Lenguaje Python

Cursada 2025

Clase 3: funciones (cont.) - expresiones lambda

Seguimos hablando de funciones

```
def my_function(param1, param2):
        sentences
        return <expression>
```

- ¿Necesitamos declarar los parámetros de la función?
- La sentencia **return es opcional**. ¿Qué pasa si no la incluimos?

Uno de los ejemplos de la clase pasada:

In [ ]:
```
movies = [50, 48, 85, 93, 100]

def average_calculation(movies_duration):
    """ This function calculates the average of the lengths of the movies received by
    movies_duration: is a list with the duration in minutes of the movies
    """
    total_movies = len(movies_duration)
    average_movies = 0 if total_movies == 0 else sum(movies_duration) / total_movies
    return average_movies
```

In [ ]:
```
average_calculation(movies)
```

Tenían una tarea: ¿de qué forma se pasan los parámetros en Python?

Parámetros en Python

Veamos un ejemplo más sencillo: ¿qué podemos observar al ejecutar el código?

In [ ]:
```
def my_demo_function1(param):
    """This function modifies the received parameter."""

    print(f"El valor de param AL INGRESAR a la función es {param}")
    param = 0
    print(f"El valor de param DENTRO de la función es {param}")
```

In [ ]:
```
number = 10
my_demo_function1(number)
print(f"LUEGO de invocar a la función el valor de num es {number}")
```

Y ahora analicemos este otro ejemplo:

In [ ]:
```
def my_demo_function2(param):
    """This function updates the first position of the list received as a parameter."
```

```
    print(f"El valor de param AL INGRESAR a la función es {param}")
    param[0] = "cero"
    print(f"El valor de param DENTRO de la función es {param}")
```

In [ ]:
```
my_list = [100, 200, 300]
my_demo_function2(my_list)
print(f"LUEGO de invocar a la función el valor de lista es {my_list}")
```

Entonces, ¿qué podemos decir sobre el pasaje de parámetros en Python?

Cuando pasamos un parámetro a una función, pasamos una copia de la referencia al objeto pasado

Miremos este otro ejemplo

In [ ]:
```
def my_demo_function3(param):
    """This function updates the first position of the list received as a parameter."

    new_list = param[:]
    print(f"El valor de param AL INGRESAR a la función parametros_colecciones es {new_
    new_list[0] = "cero"
    print(f"El valor de param DENTRO de la función parametros_colecciones es {new_lis
```

In [ ]:
```
my_list = [100, 200, 300]
my_demo_function3(my_list)
print(f"LUEGO de invocar a la función el valor de lista es {my_list}")
```

¿Qué pasa? ¿Trabajamos con una copia?

Atención! Recordemos qué pasa estructuras que contienen elementos indexados.

¿Podemos retornar más de un valor?

DESAFÍO 1

> Queremos definir una función que, dada una cadena de caracteres, retorne la **cantidad de vocales abiertas, vocales cerradas y la cantidad total de caracteres** de la misma.

Una posible solución

¿Qué tipo de dato retorna la función?

In [ ]:
```
def process_sentence(sentence):
    """ This function ... """

    sentence = sentence.lower()
    aeo = sentence.count("a") + sentence.count("e") + sentence.count("o")
```

```
    iu = sentence.count("i") + sentence.count("u")
    return aeo, iu, len(sentence)
```

In [ ]:
```
my_sentence = process_sentence("Seminario de Python")
type(my_sentence)
```

¿Cómo accedemos a los valores retornados?

- En el **return** se devuelve una tupla, por lo tanto, accedemos como en cualquier tupla:

In [ ]:
```
open_vowels = process_sentence("Seminario de Python")[0]
open_vowels
```

In [ ]:
```
open_vowels, closed_vowels, len_sentence = process_sentence("Espero que deje de llove
open_vowels
```

Veamos algo muy interesante

Observemos esta estructura: ¿de qué tipo es?

In [ ]:
```
music = {"bart": {"internacional": ["AC/DC", "Led Zeppelin", "Bruce Springsteen"],
                  "nacional": ["Pappo", "Miguel Mateos", "Los Piojos", "Nonpalidece"]
                  },
         "lisa": {"internacional": ["Ricky Martin", "Maluma"],
                  "nacional": ["Lali", "Tini", "Wos"]
                 }
          }
```

Ahora observemos esta función:

En particular, analicemos la lista de argumentos. ¿Notan algo?

In [ ]:
```
def my_music(music, name, music_type="nacional"):
    """This function ... """
    if name in music:
        user = music[name]
        for elem in user[music_type]:
            print(elem)
    else:
        print(f"¡Hola {name}! No tenés registrada música en esta colección")
```

In [ ]:
```
my_music(music, "lisa", "internacional")
```

Python permite definir parámetros con valores por defecto

> Si hay más de un argumento, los que tienen **valores por defecto siempre van al final de la lista de parámetros**.

In [ ]:
```
def demo_function(param1, param2 = "Hola"):
    print(f"{param1 = } - {param2 = }")
```

In [ ]:
```
demo_function(10)
```

> Los parámetros formales y reales se asocian de acuerdo al **orden posicional**, pero es posible invocar a la función con los parámetros en **otro orden** pero **nombrando al parámetro**.

```
def my_music(music, name, music_type="nacional"):
    """This function ... """
```

In [ ]:
```
my_music(music, music_type="internacional", name="lisa")
```

¿Recuerdan este ejemplo? Observemos el print

In [ ]:
```
word = "casa"
for letter in word:
    print(letter, end=" ")
```

In [ ]:
```
help(print)
```

Un último ejemplo

In [ ]:
```
processing_type = "invertido"
```

In [ ]:
```
def show_string(sentence, order=processing_type):
    """ This function returns the string according to the order parameter """

    return sentence[::-1] if order == "invertido" else sentence[:]
```

In [ ]:
```
show_string("Hola")
```

In [ ]:
```
processing_type = "normal"
```

¿Qué pasa ahora si vuelvo a invocar a la función?

In [ ]:
```
show_string("Hola")
```

Los valores por defecto de los parámetros se evalúan una única vez cuando se define la función.

DESAFÍO 2

> Queremos escribir una función que imprima sus argumentos agregando de qué tipo son.

¿Qué tiene de distinta esta función respecto a las que vimos antes o conocemos de otros lenguajes? Por ejemplo:

```
my_args(1)          --> 1 es de tipo <class 'int'>
my_args(2, "hola") --> 2 es de tipo <class 'int'>,
                       hola es de tipo <class 'str'>
my_args([1,2], "hola", 3.2)
                    --> [1, 2] es de tipo <class 'list',
```

```
hola es de tipo <class 'str'>,
3.2 es de tipo <class 'float'>
```

¿Opciones?

In [ ]:
```
def my_args():
    print("Hola")
def my_args(par1, par2):
    print(par1)
def my_args(par1, par2, par3):
    print(par1)

my_args()
```

Esto no funciona.

Podemos definir funciones con un número variable de parámetros

In [ ]:
```
def my_args(*args):
    """ This function..."""

    for value in args:
        print(f"{value} es de tipo {type(value)}")
```

- **args** es una **tupla** que representa a los parámetros pasados.

In [ ]:
```
my_args(1)
print("-"*30)
my_args(2, "hola")
print("-"*30)
my_args([1,2], "hola", 3.2)
```

Otra forma de definir una función con un número variable de parámetros

In [ ]:
```
def my_args1(**kwargs):
    """ This function ..... """

    for key, value in kwargs.items():
        print(f"{key} es {value}")
```

- **kwargs** es una **diccionario** que representa a los parámetros pasados.

In [ ]:
```
my_args1(band1= 'Nirvana', band2="Foo Fighters", band3="AC/DC")
```

También podemos tener lo siguiente:

In [ ]:
```
def show_data(par1, par2, par3):
    print(par2)
```

```
sequence = (1, 2, 3)
show_data(*sequence)
```

- ¿De qué tipo es **sequence**?
- **CONSIGNA:** probar en casa si es posible utilizar otras secuencias vistas.

Otra forma

In [ ]:
```
def show_contact(name, phone):
    #print(type(phone))
    print(name, phone)

contact = {"name": "Messi", "phone": 12345}
show_contact(**contact)
```

- ¿De qué tipo es **contact**?
- Observar el nombre de los parámetros: ¿qué podríamos decir?

CONSIGNA: probar en casa estos ejemplos:

In [ ]:
```
def show_elements1(param1, param2, param3, param4):
    """Imprimo los valores de los dos primeros parámetros"""
    print( f"{param1}, {param2}")

def show_elements2(*arguments):
    """Imprimo los valores de los argumentos"""
    for value in arguments:
        print( value)

def show_elements3(**arguments):
    """Imprimo una tabla nombre-valor"""
    for name, value in arguments.items():
        print( f"{name} = {value}")
```

In [ ]:
```
numbers = { "uno": 1, "dos": 2, "tres":3, "cuatro": 4}

print("Invoco a show_elements3 con  numbers como parámetro")
show_elements3(**numbers)
print("-" * 20)

print("Invoco a show_elements3 con los parámetros nombrados")
show_elements3(param1 =1, param2 = 2, param3 = 3, param4 = 4)
print("-" * 20)

print("Invoco a show_elements1 con  parámetros nombrados")
show_elements1(param1 ="I", param2 = "II", param3 = "III", param4 = "IV")
```

In [ ]:
```
print("Invoco a show_element1 con  parámetros simples")
show_elements1("I", "II", "III", "IV")

print("-" * 20)

print("Invoco a show_elements2 con  parámetros simples")
show_elements2(1, 2, 3, 4)
```

CONSIGNA: ¿todo junto se puede?

> Probar este ejemplo y analizar el orden en el que definimos los parámetros.

In [ ]:
```
def show_more_values(initial_message, *in_other_language, **in_detail):
    print("Mensaje original")
    print(initial_message)

    print("\nEn otros idiomas")
    print("-" * 40)
    for value in in_other_language:
        print(value)
    print("\nEn detalle")
    print("-" * 40)

    for key in in_detail:
        print(f"{key}: {in_detail[key]}")
    print("\nFuente: traductor de Google. ")
```

In [ ]:
```
show_more_values("Hola",
    "hello", "Hallo", "Aloha ", "Witam", "Kia ora",
    ingles= "hello",
    aleman="Hallo",
    hawaiano="Aloha",
    polaco="Witam",
    maori="Kia ora")
```

Variables locales y globales

In [ ]:
```
x = 12
a = 13
def my_function(a):
    #global x
    x = 9
    a = 10

my_function(a)
print(a)
print(x)
```

- Variables locales enmascaran las globales.
- Acceso a las globales mediante **global**.

No es una buena práctica utilizar variables globales

Espacio de nombres

> Un espacio de nombres **relaciona nombres con los objetos que referencian**.

- Cuando se invoca a una función, se crea un espacio de nombres **local** con todos los recursos definidos en la función y luego se elimina cuando la función finaliza su ejecución.

Volveremos a este tema más adelante...

ATENCION: ¿qué pasa en los siguientes ejemplos?

In [ ]:
```
x = 12

def my_function1():
    temp = x + 1
    print(temp)

def my_function2():
    x = x + 1
    print(x)
```

In [ ]:
```
my_function1()
```

¿Cuál es el problema al invocar a my_function2?

Python permite definir funciones anidadas

In [ ]:
```
def one():
    def one_one():
        print("uno_uno")
    def one_two():
        print("uno_dos")

    print("uno")
    one_one()

def two():
    print("dos")
    one_two()

one()
```

¿Cuál es el problema al invocar a la función two?

¿Qué imprimimos en este caso?

In [ ]:
```
x = 0
def one():
    def one_one():
        #nonlocal x
        #global x
        x = 100
        print(f"En one_one: {x= }")

    x = 10
    one_one()
    print(f"En one: {x= }")

one()
print(f"En ppal: {x= }")
```

- **global** y **nonlocal** permiten acceder a variables no locales a una función.

Recordemos el Zen de Python ...

```
...
Simple es mejor que complejo.
...
Plano es mejor que anidado.
...
Espaciado es mejor que denso.

La legibilidad es importante.
...
```

¿Entonces?

Observemos este ejemplo

In [ ]:
```python
def send_message(message, to_person, from_person="alguien", subject="Consulta"):
    """ This function displays a message...
    Arguments:
    to_person: string that represents the recipient of the message
    from_person: string that represents  the person who sends the message
    subject: that represents the subject of the message
    """

    print(f"""
            Origen: {from_person}
            Destino: {to_person}
            Asunto: {subject}
            Mensaje: {message}
        """)

send_message("¿Cuánto falta para que temine la clase?", "Profe")
```

Atributos de las funciones

> Las funciones en Python también son **objetos**.

- Algunos de sus atributos:
  - **my_function.__doc__**: es el **docstring**.
  - **my_function.__name__**: es una cadena con el nombre la función.
  - **my_function.__defaults__**: es una tupla con los valores por defecto de los parámetros opcionales.

In [ ]:
```python
print(send_message.__doc__)
print(send_message.__defaults__)
print(send_message.__name__)
```

DESAFÍO 3

> Queremos implementar una función que dada una cadena de texto, retorne las palabras que contiene en orden alfabético.

In [ ]:
```
# Una posible solución
def sort1(sentence):
    """ Example using method sort"""

    words = sentence.split()
    #words.sort(key=str.lower)
    words.sort()
    return words
```

In [ ]:
```
print(sort1("Hoy puede ser un gran día. "))
```

¿De qué tipo es el valor enviado como parámetro a **key**?

Otra forma

In [ ]:
```
def sort2(sentence):
    """ Example using function sorted"""

    words = sentence.split()
    return sorted(words, key=str.lower)
```

In [ ]:
```
print(sort2("Hoy puede ser un gran día. "))
```

DESAFÍO 4

> Queremos implementar una función que, dada una lista de canciones,

nos retorne la lista ordenada de acuerdo al intérprete. Tenemos registrada la siguiente info: **tema, intérprete, año, categoría.**

In [ ]:
```
my_music = [
        ("Bohemian Rapsody",  "Queen", "1975", "rock"),
        ("Thunder Road", "Bruce Springsteen", "1975", "rock"),
        ("Imagine", "John Lennon", "1971", "rock"),
        ("Locura y Realidad", "Las Pastillas del Abuelo", "2007", "rock"),
        ("Ascenso", "Jauría", "2010", "rock"),
        ]
```

In [ ]:
```
# Posible solución
```

Analicemos esta solución

In [ ]:
```
def sort3(music):
    """ This function .... """

    return sorted(music, key=lambda elem: elem[1])
```

In [ ]:
```
for elem in sort3(my_music):
    print(elem)
```

RECORDEMOS: **key** espera una función.

¿Qué son las expresiones lambda?

- Son funciones anónimas.

```
lambda args : expression
```

- +Info

In [ ]:
```
lambda a, b:  a+b
lambda a, b=1: a+b
```

In [ ]:
```
lambda a, b=1: a+b

def my_sum(a, b=1):
    return a+b
```

Algunos ejemplos de uso

In [ ]:
```
actions_list = [lambda x: x*2, lambda x: x*3]
```

¿Qué tipo de elementos contiene la lista?

In [ ]:
```
param = 4

for action in actions_list:
    print(action(param))
```

Un ejemplo de la documentación oficial

In [ ]:
```
def make_incrementor(n):
    return lambda x: x+n

f = make_incrementor(2)
g = make_incrementor(6)
```

In [ ]:
```
print(f(42), g(42))
print(make_incrementor(22)(33))
```

¿Cuál es el tipo de **f** y **g**? ¿Por qué es correcto el último **print**?

Veamos algunos usos comunes de estas expresiones

La función map

In [ ]:
```
def double(x):
    return 2*x
```

In [ ]:
```
numbers = [1, 2, 3, 4, 5, 6, 7]
doubles = map(double, numbers)
for elem in doubles:
    print(elem, end="  ")
```

La función filter

In [ ]:
```
def is_even(x):
    return x%2 == 0
```

In [ ]:
```
numbers  = [1, 2, 3, 4, 5, 6, 7]
even_numbers = filter(is_even, numbers)

for elem in even_numbers:
    print(elem, end="  ")
```

map y filter con lambda

In [ ]:
```
numbers  = [1, 2, 3, 4, 5, 6, 7]

double_numbers = map(lambda x: 2*x, numbers)
even_numbers = filter(lambda x: x%2 == 0, numbers)
```

In [ ]:
```
for elem in double_numbers:
    print(elem, end="  ")
```

La función reduce

In [ ]:
```
from functools import reduce

numbers  = [1, 2, 3, 4, 5, 6, 7]

reduce(lambda a, b: a + b, numbers)
```

¿Qué dice la PEP 8 respecto al uso de lambda?

Si queremos definir la función **double**, por ejemplo, usemos **def** y no **lambda**.

```
# Si
def double(x):
    return 2*x

# No
double = lambda x: 2*x
```

Y en estos casos...

In [ ]:
```
numbers = [1, 2, 3, 4]

doubles = [2*x for x in numbers]
#doubles = map(lambda x: 2*x, numbers)

for elem in doubles:
    print(elem, end="  ")
```

In [ ]:
```
mumbers = [1, 2, 3, 4]

pares = [x for x in numbers if x%2 ==0]
#pares = filter(lambda x: x%2 == 0, numbers)
```

```
for elem in pares:
    print(elem, end="  ")
```

In [ ]:
```
from functools import reduce

numbers  = [1, 2, 3, 4, 5, 6, 7]
sum(numbers)

#reduce(lambda a, b: a + b, numbers)
```

Entonces...¿usamos lambda?

De los ejemplos vistos en la clase...

```
def make_incrementor(n):
    return lambda x: x+n

def sort3(music):
    """ Usamos sorted con una expresión lambda"""

    return sorted(music, key=lambda elem: elem[0])
```

Un artículo sobre estilo de código

https://realpython.com/python-pep8/

CONSIGNA: tarea para el hogar

> Queremos codificar una frase según el siguiente algoritmo:

```
encripto("a") --> "b"
encripto("ABC") --> "BCD"
encripto("Rock2021") --> "Spdl3132"
```

Una explicación simple de la Wikipedia: Cifrado César

- Escribir dos versiones de la solución: una sin usar lambda y otra usando.
- Si quieren, subir el código modificado a su repositorio en GitHub y compartir el enlace a la cuenta @clauBanchoff Y @sofiamartin.

Seguimos la semana próxima ...